Lecture 26

Victory Lap!

Course Goal

9 weeks to learn *some fundamental concepts* of Programming Languages

- Become better programmers, even in languages we won't use
- Learn to distinguish surface differences from deeper principles
- Develop a systematic methodology for what programs mean
- Pick up some technical jargon to support further study

Mindset

- It’s too easy to forget how mind-bending programming first was
  - *“When you're thinking about something that you don't understand, you have a terrible, uncomfortable feeling called confusion.”*
  - Learning new things often means working through confusion: that’s OK!!
- It’s too easy to think everything is a nail and we only need hammers
  - *“We become what we behold. We shape our tools and then our tools shape us.”*
  - Learning diverse ideas makes us robust when facing new problems!

The Recipe: Syntax + Semantics

- Syntax: how programs are written down
  - Roughly “spelling and grammar”
- Semantics: what programs ***mean***
  - Type checking : compile time (aka “static”) semantics
  - Evaluation : run time (aka “dynamic”) semantics
- Both syntax and semantics matter
  - 341’s view ≈ “Semantics give the essence of a PL.”
  - 341’s view ≈ “Syntax is more about taste.”

Phase 1: Learning OCaml

Phase 1: Learning OCaml

- Intro to FP (expressions, variants, first-class functions, etc)
- “Operational Semantics”

```
Inductive cmd :=
| Skip
| Assign (x : var) (e : arith)
| Sequence (c1 c2 : cmd)
| If (e : arith) (then_ else_ : cmd)
| While (e : arith) (body : cmd).
```

$$\frac{}{(v,\ \mathtt{Skip}) \Downarrow v}\ \text{EvalSkip}$$

$$\frac{v' = v[x \mapsto [\![e]\!]_v]}{(v,\ x := e) \Downarrow v'}\ \text{EvalAssign}$$

$$\frac{(v,\ c_1) \Downarrow v' \qquad (v',\ c_2) \Downarrow v''}{(v,\ c_1; c_2) \Downarrow v''}\ \text{EvalSeq}$$

$$\frac{[\![e]\!]_v \neq 0 \qquad (v,\ c_1) \Downarrow v'}{(v,\ \mathtt{if}\ e\ c_1\ c_2) \Downarrow v'}\ \text{EvalIfT}$$

$$\frac{[\![e]\!]_v = 0 \qquad (v,\ c_2) \Downarrow v'}{(v,\ \mathtt{if}\ e\ c_1\ c_2) \Downarrow v'}\ \text{EvalIfF}$$

$$\frac{[\![e]\!]_v \neq 0 \qquad (v,\ c;\ \mathtt{while}\ e\ c) \Downarrow v'}{(v,\ \mathtt{while}\ e\ c) \Downarrow v'}\ \text{EvalWhileT}$$

$$\frac{[\![e]\!]_v = 0}{(v,\ \mathtt{while}\ e\ c) \Downarrow v}\ \text{EvalWhileF}$$

Phase 2: Implementing Interpreters

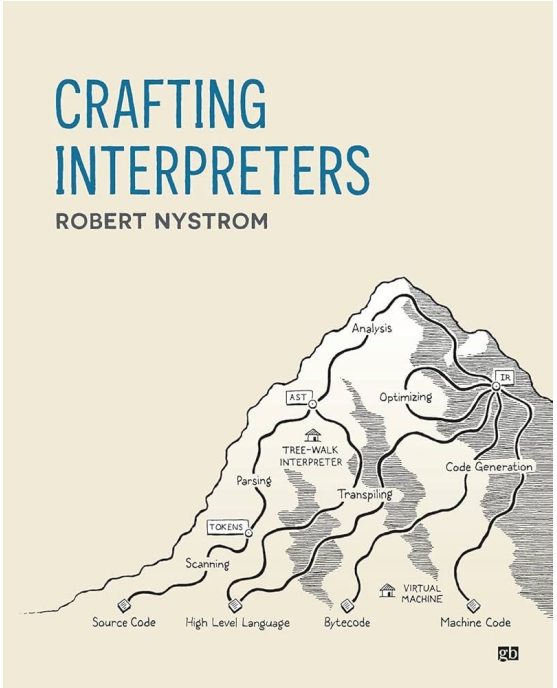
Revisit each language feature and see how it works under the hood

Trefoil has the core features of any good functional programming language

What’s missing from Trefoil?

Where to go from here? Learning more PL

- Compilers
- Parsers
- Fast interpreters
- Domain-specific languages



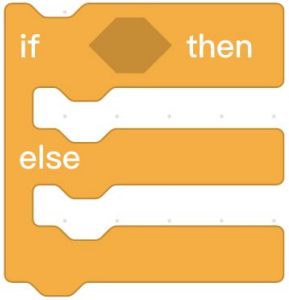
Where to go from here? Languages are everywhere!

A simple configuration language

Let's add some abstractions and parameters.

Let's make the abstractions compose recursively.

```
{{- if (include "kafka.createJaasSecret" .) }}
apiVersion: v1
kind: Secret
metadata:
  name: {{ template "kafka.fullname" . }}-jaas
  labels: {{- include "common.labels.standard" . | nindent 4 }}
    {{- if .Values.commonLabels }}
    {{- include "common.tplvalues.render" ( dict "value" .Values
    {{- end }}
  {{- if .Values.commonAnnotations }}
  annotations: {{- include "common.tplvalues.render" ( dict "val
  {{- end }}
type: Opaque
data:
  {{- if (include "kafka.client.saslAuthentication" .) }}
  {{- $clientUsers := coalesce .Values.auth.sasl.jaas.clientUser
  {{- $clientPasswords := coalesce .Values.auth.sasl.jaas.client
  {{- if $clientPasswords }}
  client-passwords: {{ join "," $clientPasswords | b64enc | quot
  {{- else }}
  {{- $passwords := list }}
  {{- range $clientUsers }}
  {{- $passwords = append $passwords (randAlphaNum 10) }}
  {{- end }}
  client-passwords: {{ join "," $passwords | b64enc | quote }}
  {{- end }}
  {{- end }}
  {{- $zookeeperUser := coalesce .Values.auth.sasl.jaas.zookeepe
  {{- if and .Values.zookeeper.auth.enabled $zookeeperUser }}
  {{- $zookeeperPassword := coalesce .Values.auth.sasl.jaas.zook
  zookeeper-password: {{ default (randAlphaNum 10) $zookeeperPas
  {{- end }}
  {{- if (include "kafka.interBroker.saslAuthentication" .) }}
  {{- $interBrokerPassword := coalesce .Values.auth.sasl.jaas.ir
  inter-broker-password: {{ default (randAlphaNum 10) $interBrok
  {{- end }}
{{- end }}
```



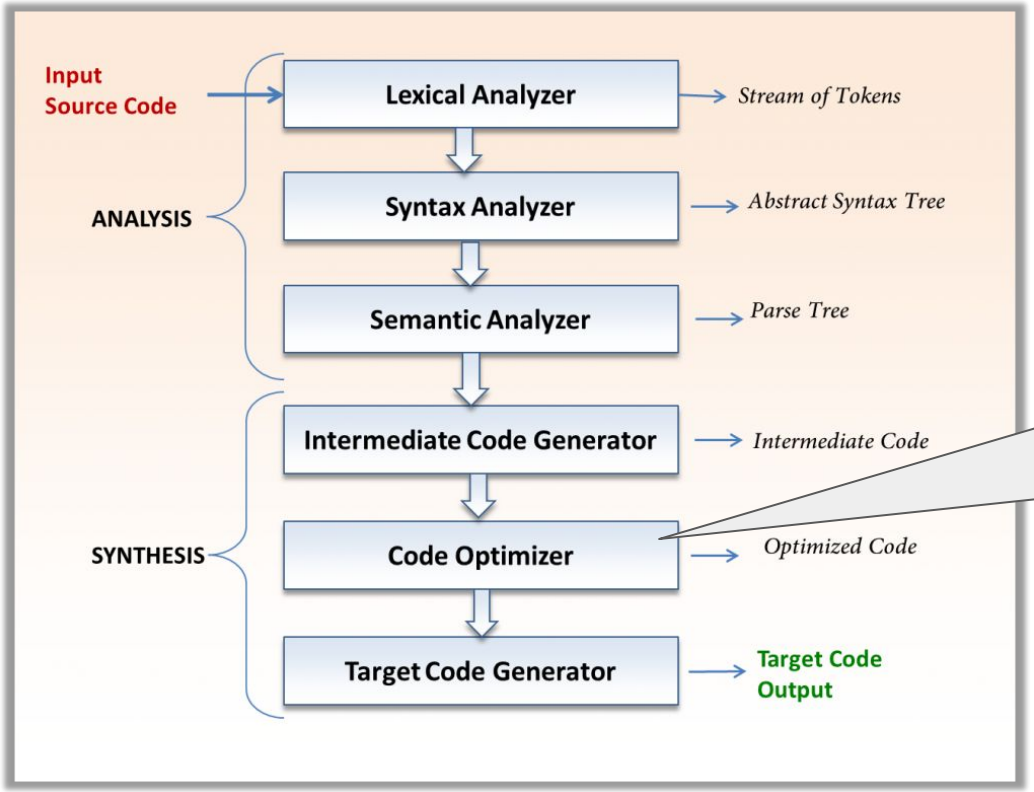
Parsing

```
IfThenElse of expr * stmt * stmt
IfThen of expr * stmt
```

```
if 1 + 2 < 4 then foo(1 + 2) else bar()
if 5 = 10 then foo(bar(x))
```

```
(if (< (+ 1 2) 4) (foo (+ 1 2)) (bar))
(if (= 5 10) (foo (bar x)))
```

Compilers



Lots of sequential transformations over the IR (basically an AST)

Where to go from here? Learning more languages

- Rust
- Haskell
- Datalog
- Agda
- Rocq

FEBRUARY 26, 2024

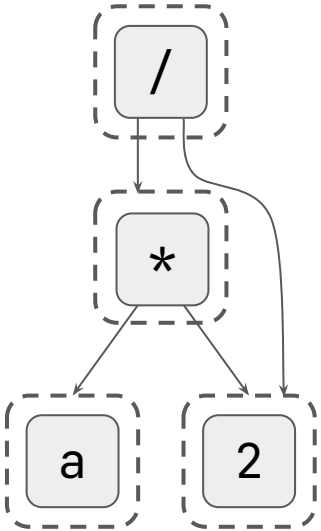
Press Release: Future Software Should Be Memory Safe

ONCD › BRIEFING ROOM › PRESS RELEASE

Where to go from here? PL Research

- Applying PL ideas / mindset to other domains
  - Knitting, 3D printing, graphics
- New languages + language features
  - Filament, Cyclone
- Program Synthesis
  - Lakeroad, Enumo
- Program Analysis
  - Optional Checker

Equality Saturation



Equality Saturation